
Private Text Generation with RNM-based Decoding

S. Samrudh Raaj
Department of DSAI
Indian Institute of Technology, Madras
da24b025@smail.iitm.ac.in

Raghav Iyengar
Department of DSAI
Indian Institute of Technology, Madras
da24b020@smail.iitm.ac.in

Rohan Sai Pabba
Department of DSAI
Indian Institute of Technology, Madras
da24b049@smail.iitm.ac.in

Abstract

As large language models (LLMs) are increasingly used for tasks involving sensitive contexts, ensuring differential privacy (DP) during text generation is a significant hurdle. Current state-of-the-art methods, like INVISIBLEINK, achieve DP by treating the LLM’s next-token selection as an application of the Exponential Mechanism on the model’s output scores (logits). This is mathematically the same as using the Report Noisy Max (RNM) rule with Gumbel noise, which results in standard softmax probabilities. However, the potential benefits of using different types of noise distributions with RNM haven’t been fully investigated. This project aims to test the effectiveness of RNM decoding when using exponential noise and other such noise mechanisms which have DP guarantees. Since RNM with exponential noise creates a distinct probability distribution compared to standard softmax sampling, it might offer better compatibility with decoding parameters such as temperature and top-k sampling, potentially leading to improved semantic utility.

1 Introduction

Large Language Models exhibit powerful reasoning and synthesis capabilities, often leaking private or sensitive context via Retrieval-Augmented Generation (RAG) (Zeng et al. [10]). To prevent this leakage of sensitive data in the generated output, Differential Privacy (DP) [6] offers rigorous, mathematically provable guarantees. Existing DP text generation methods, including those by Amin et al. [2] and the highly scalable INVISIBLEINK [1] framework, primarily rely on the Exponential Mechanism [3, 6]. In practice, this is equivalent to applying standard softmax sampling to the logits or utilizing the Gumbel-Max trick [4]. However, the Exponential Mechanism is not the only way to achieve DP selection. The Report Noisy Max (RNM) algorithm offers a generalized framework where different noise distributions can be applied to the logits before selection [5]. This project introduces RNM with exponential noise as an alternative to the standard Exponential Mechanism, aiming to rigorously evaluate its impact on synthetic text utility and decoding efficiency.

2 Motivation

The primary motivation for this work comes from the mathematical equivalence between the standard Exponential Mechanism and RNM with Gumbel noise. Since they yield the exact same softmax probability distribution, prior DP decoding methods often suffer from the same well-known issues of softmax sampling under high noise i.e. degenerate or incoherent text due to a flattened probability distribution.

By substituting Gumbel noise with exponential noise in the RNM framework, we alter the shape of the resulting probability distribution. The sharp probability peak at zero or the "zero-mode" characteristic of exponential noise would favor high-logit tokens more strongly and might interact uniquely with decoding parameters like temperature scaling (τ) and top- k truncation. We hypothesize that RNM with exponential noise might better suppress unreliable low-probability tokens and select higher-scoring tokens more frequently (as shown in Figure 1) under DP constraints, which would lead to higher-quality private text generation without the severe utility degradation seen in traditional softmax-based DP decoding.

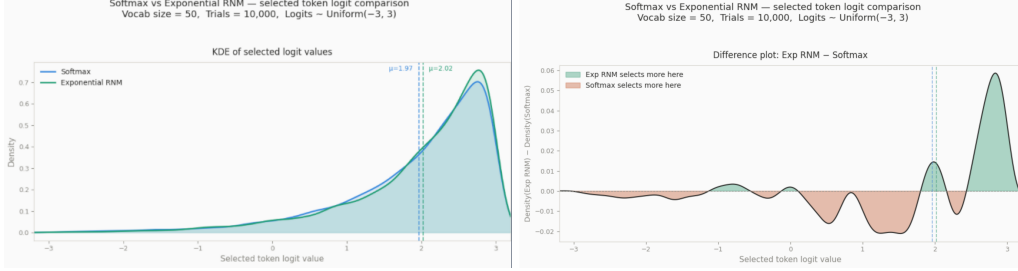


Figure 1: Difference between selection frequencies for different logit values

3 Proving ϵ -DP guarantee for RNM Method with Exponential Noise

L_∞ **Sensitivity:** Let $\bar{\phi}$ denote the average clipped logits. The L_∞ sensitivity Δ is defined as

$$\Delta = \max_{R \simeq R'} \|\bar{\phi}(R) - \bar{\phi}(R')\|_\infty$$

$$\Delta = \max_{i \in V_+^k} \max_{R \simeq R'} |\bar{\phi}(i, R) - \bar{\phi}(i, R')|$$

Here R, R' are two adjacent sets of private references and they follow replace-by-null adjacency notion [1].

From the sensitivity definition, the following properties will be satisfied:

1. $\bar{\phi}(i, R) \leq \bar{\phi}(i, R') + \Delta$
2. $-\bar{\phi}(i, R) \leq -\bar{\phi}(i, R') + \Delta$

The RNM algorithm outputs an index i such that $i = \arg \max_j (\bar{\phi}(j, R) + Z_j)$, where Z_j are i.i.d. random variables following an Exponential distribution $Z_j \sim \text{Exp}(\lambda)$.

For the value selected to be i , we need Z_i to be greater than or equal to the threshold for i to be maximum. Say that is called T_R . We know

$$T_R = \max_{j \neq i} \{\bar{\phi}(j, R) + Z_j - \bar{\phi}(i, R)\}$$

For the adjacent dataset R' , the threshold will be $T_{R'}$ and from properties (1) and (2), we can conclude that:

$$T_{R'} - 2\Delta \leq T_R \leq T_{R'} + 2\Delta$$

Hence the privacy loss will be as follows:

$$\text{Loss} = \log \left(\frac{P(i \text{ is selected} | R)}{P(i \text{ is selected} | R')} \right) = \log \left(\frac{e^{\frac{-T_R}{\lambda}}}{e^{\frac{-T_{R'}}{\lambda}}} \right) = \frac{T_{R'} - T_R}{\lambda}$$

We can now say that

$$\frac{-2\Delta}{\lambda} \leq \text{Loss} \leq \frac{2\Delta}{\lambda}$$

Hence we know that the Loss is bounded and that gives us $\epsilon = \frac{2\Delta}{\lambda}$

Composition: We can say that the RNM with Exponential Noise is $\rho - zCDP$ with $\rho = \frac{\epsilon^2}{2}$. Now to compose this over T tokens, we can say that $\rho_{total} = T\rho = T\frac{\epsilon^2}{2} = \frac{2T\Delta^2}{\lambda^2}$

We can then convert this $\rho_{total} - zCDP$ to $(\epsilon', \delta) - DP$ using the following relation:

$$\epsilon' = \rho_{total} + \sqrt{4\rho_{total} \log\left(\frac{1}{\delta}\right)}$$

4 Tail Bounds and Expected Value of Suboptimality for RNM Method with Exponential Noise

Suboptimality Bound: The suboptimality of the algorithm is the difference between the optimal score and the score of the selected index: $OPT - \bar{\phi}(i, R)$ (Here score refers to the average clipped logits and selected index is the index obtained from RNM with Exponential noise). We seek to bound how large this error can get (Tail Bound) and find its average case (Expected Value).

4.1 Tail Bound Derivation

We are bounding $P(OPT - \bar{\phi}(i, R) > t)$ for $t > 0$. For this suboptimal index i to be chosen as the maximum, its noisy score must exceed the noisy score of the optimal index i^* :

$$\begin{aligned}\bar{\phi}(i, R) + Z_i &\geq \bar{\phi}(i^*, R) + Z_{i^*} \\ \bar{\phi}(i, R) + Z_i &\geq OPT + Z_{i^*}\end{aligned}$$

As the exponential noise is strictly non-negative ($Z_{i^*} > 0$), it implies

$$\bar{\phi}(i, R) + Z_i \geq OPT \implies Z_i \geq OPT - \bar{\phi}(i, R) > t$$

For the algorithm to make an error greater than t , there must exist *at least one* suboptimal element $j \in \mathcal{R}$ where its noise Z_j is high enough to compensate for its low score. As an upper bound, we can take the sum of such probabilities.

$$\begin{aligned}P(\text{Suboptimality} > t) &\leq P\left(\bigcup_{j: \bar{\phi}(j, R) < OPT - t} \{Z_j \geq OPT - \bar{\phi}(j, R)\}\right) \\ P(\text{Suboptimality} > t) &\leq \sum_{j: \bar{\phi}(j, R) < OPT - t} P(Z_j \geq OPT - \bar{\phi}(j, R))\end{aligned}$$

For an Exponential distribution $Z_j \sim \text{Exp}(\lambda)$, the cumulative tail probability is $P(Z_j \geq x) = e^{-x/\lambda}$.

$$P(\text{Suboptimality} > t) \leq \sum_{j: \bar{\phi}(j, R) < OPT - t} e^{-\frac{OPT - \bar{\phi}(j, R)}{\lambda}}$$

We know that every term of the sum has a sub-optimality bound of atleast t and there exists atmost $|V_k^+| - 1$ elements (where V_k^+ represents the Expanded Top-K+ Vocabulary [1]) which can satisfy this criterion, we get

$$P(\text{Suboptimality} > t) \leq (|V_k^+| - 1)e^{-\frac{t}{\lambda}} < |V_k^+|e^{-\frac{t}{\lambda}}$$

To find the confidence bound, let us assume the bound to be delta.

$$\delta = |V_k^+|e^{-\frac{t}{\lambda}} \implies e^{-\frac{t}{\lambda}} = \frac{|V_k^+|}{\delta} \implies t = \lambda \ln\left(\frac{|V_k^+|}{\delta}\right)$$

Substituting $\lambda = \frac{2\Delta}{\epsilon}$ (from the privacy loss bound), we get that with probability at least $1 - \delta$, the suboptimality is bounded by **Tail Bound**:

$$\text{Suboptimality} \leq \frac{2\Delta}{\epsilon} \ln\left(\frac{|V_k^+|}{\delta}\right)$$

4.2 Expected Value of Suboptimality

Let X represent the suboptimality error. As X is a non-negative random variable, the expected value can be computed by integrating its tail probability:

$$E[X] = \int_0^\infty P(X > t) dt$$

We have derived that $P(X > t) \leq |V_k^+| e^{-t/\lambda}$. However, as probabilities cannot exceed 1 we can use piece-wise bound $P(X > t) \leq \min(1, |V_k^+| e^{-t/\lambda})$. We have to split the integral into two parts, transitioning where $1 = |V_k^+| e^{-t/\lambda}$. This occurs at $t = \lambda \ln |V_k^+|$ and gives us

$$E[X] \leq \int_0^{\lambda \ln |V_k^+|} 1 dt + \int_{\lambda \ln |V_k^+|}^\infty |V_k^+| e^{-\frac{t}{\lambda}} dt$$

Evaluating the two integrals:

1. $\int_0^{\lambda \ln |V_k^+|} 1 dt = \lambda \ln |V_k^+|$
2. $\int_{\lambda \ln |V_k^+|}^\infty |V_k^+| e^{-\frac{t}{\lambda}} dt = |V_k^+| \left[-\lambda e^{-\frac{t}{\lambda}} \right]_{\lambda \ln |V_k^+|}^\infty = |V_k^+| \left(0 - \left(-\lambda e^{-\ln |V_k^+|} \right) \right) = |V_k^+| \left(\lambda \cdot \frac{1}{|V_k^+|} \right) = \lambda$

Adding these together gives:

$$E[X] \leq \lambda \ln |V_k^+| + \lambda$$

$$E[X] \leq \lambda (\ln |V_k^+| + 1)$$

Substituting for $\lambda = \frac{2\Delta}{\epsilon}$, we get that the **Expected Suboptimality** is bounded by:

$$E[\text{Suboptimality}] \leq \frac{2\Delta}{\epsilon} (\ln |V_k^+| + 1)$$

Both the suboptimality bound and expected suboptimality scale linearly with the noise parameter λ , with a multiplicative factor of $O(\ln |V_k^+|)$. Since this factor grows slowly, we can conclude that the mechanism extracts nearly the maximum possible utility for the given privacy budget ϵ or the given noise parameter λ .

Both the bounds also scale as $O(\ln |V_k^+|)$, it implies that we can expand the vocabulary set comfortably with negligible effect on selection quality.

4.3 Distribution of ranks of token selection by Gumbel and Exponential mechanisms

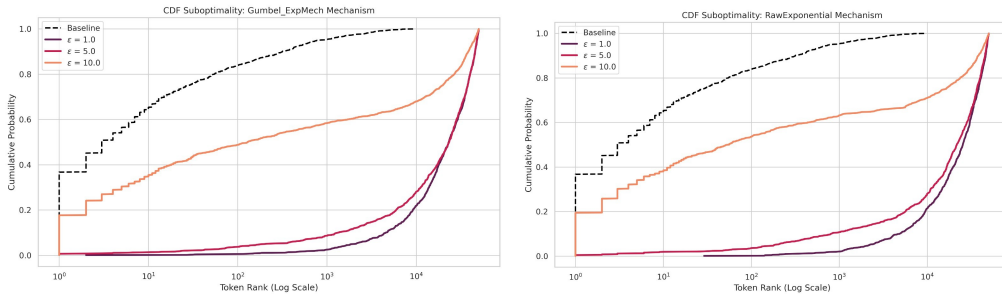


Figure 2: CDF of Rank of logit selected by Gumbel vs. Exponential Mechanism

The generated plots (Figure 2) provide a comprehensive view of how different noise distributions (Gumbel and Exponential) interact with a large vocabulary under strict sensitivity bounds. The data yields the following insights:

The Impact of the Privacy Budget (ϵ)

The CDF plots show that the baseline non private model approaches the 1 rapidly within few 100 ranks. As the privacy budget becomes more restrictive (e.g., transitioning from $\epsilon = 10.0$ to $\epsilon = 1.0$), the CDF curves bow outward to the right. At strict budgets like $\epsilon \in \{1.0, 5.0\}$, all mechanisms exhibit a near-zero probability of selecting a top-100 token, clearly showing the utility gap introduced by the noise.

Cross-Mechanism Consistency

When comparing the canonical Exponential Mechanism (implemented via exact sampling using the Gumbel-Max trick) against the application of Raw Exponential noise prior to the argmax operation, we see nearly identical utility degradation. While the Raw Exponential approach has theoretically looser suboptimality bounds than the Gumbel-Max trick, the empirical CDFs align almost perfectly under these conditions.

5 Empirical Analysis of Utility of RNM with Exponential Noise

MAUVE scores: MAUVE is a continuous evaluation metric designed to measure the distributional gap between human-written text and machine-generated text by calculating the area under a divergence frontier [8].

Perplexity Difference: Perplexity (PPL) evaluates the fluency and naturalness of generated text by measuring the inverse probability of the text sequence, normalized by the number of tokens. To measure the utility cost of applying differential privacy during decoding, we calculate the Perplexity Difference (Δ PPL).

Mechanisms:

- *Base Invisible Ink:* This is the standard Invisible Ink process of text generation with softmax-based decoding and privacy budget of $\rho - zCDP = \frac{\epsilon^2}{8}$ [1]
- *Weak Invisible Ink:* This is the modified version of Invisible Ink process to lower the privacy budget to $\rho - zCDP = \frac{\epsilon^2}{2}$ such that it is equal to the privacy budget of Invisible Ink with RNM
- *Invisible Ink + RNM:* This is the Invisible Ink process of text generation with RNM based decoding using Exponential Noise and privacy budget of $\rho - zCDP = \frac{\epsilon^2}{2}$. Here we used a batch size of 2 private samples to induce more noise (increase effective temperature) for the RNM mechanism (since it was generating very similar texts across multiple samples).

Model: We have used Llama-3.2-3B-Instruct model for generating 500 samples each of Base Invisible Ink, Weak Invisible Ink and Invisible Ink with RNM (total 1500 samples of text generated) for $\epsilon = 1, 5, 10, 20$.

Dataset: We have used the Text Anonymous Benchmark (TAB) dataset which contains European Court cases, as well the Yelp dataset which contains various human reviews (both positive and negative labelled from 0 to 4, 0 being most negative, 4 being most positive). Although we generated 500 samples each using Yelp dataset, we noticed that the plots were too noisy suggesting that we needed to scale it to more samples for a noisy dataset like Yelp (the noisy plots are not included in the figure).

We can see from [Figure 4](#) that the MAUVE scores of Invisible Ink with RNM decoding is lower than the standard softmax sampling of Invisible Ink as well as the weak Invisible Ink version. However, the Perplexity scores of Invisible Ink with RNM decoding are much closer to the Human texts than the perplexity scores of standard Invisible Ink and weak Invisible Ink with softmax sampling.

Perplexity in general is a bad metric to measure the utility of the generated text as it is a "local" metric. It is only dependent on the next token probability and has no relation to the distribution of the texts generated.

MAUVE scores, however, is a global metric as it compares the distribution of the human texts and the generated texts.

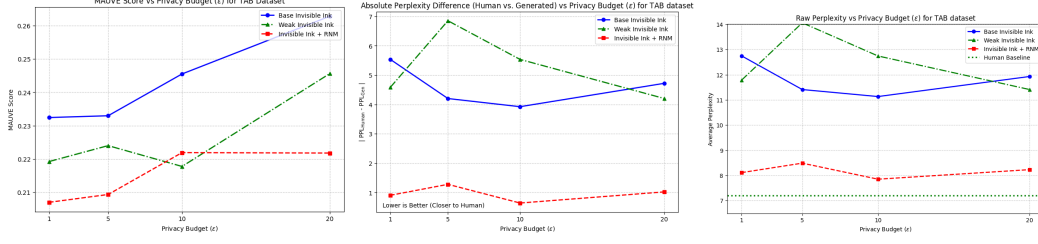


Figure 3: MAUVE scores, Perplexity differences, and raw Perplexity scores of 500 samples each of Base Invisible Ink vs. Weak Invisible Ink vs. Invisible Ink with RNM using Exponential Noise using TAB dataset

6 Loss Function Design: MLE Under the Exponential Noise Model

The empirical results in Section 5 reveal a consistent pattern: RNM-based decoding with exponential noise produces text with MAUVE scores considerably lower than that of softmax-based decoding. We now diagnose the root cause of this degradation and design a loss function that corrects it.

6.1 Fixing the Training–Decoding Mismatch

Standard language models are trained by minimizing cross-entropy loss, which is the negative log-likelihood of the softmax distribution:

$$\mathcal{L}_{\text{CE}}(\bar{\phi}_\theta, y) = -\log \frac{\exp(\bar{\phi}_y/\tau)}{\sum_{j \in V_k^+} \exp(\bar{\phi}_j/\tau)}$$

The minimizer of $\mathcal{L}_{\text{CE}}$ over model parameters θ satisfies $\text{softmax}(\bar{\phi}_\theta/\tau) = p^{\text{data}}$ in the infinite-data, infinite-capacity limit. To see why, let $q_\theta(y | x) = \text{softmax}(\bar{\phi}_\theta(x)/\tau)_y$ denote the model’s predicted distribution. The expected cross-entropy loss over the data distribution is:

$$\mathbb{E}_{(x,y) \sim p^{\text{data}}} [-\log q_\theta(y | x)]$$

This can be decomposed using the identity $\mathbb{E}_p[-\log q] = H(p) + D_{\text{KL}}(p \| q)$, where $H(p^{\text{data}})$ is the entropy of the data distribution (a constant independent of θ) and $D_{\text{KL}}(p^{\text{data}} \| q_\theta)$ is the KL divergence:

$$\mathbb{E}_{(x,y) \sim p^{\text{data}}} [-\log q_\theta(y | x)] = \underbrace{H(p^{\text{data}})}_{\text{constant in } \theta} + \underbrace{D_{\text{KL}}(p^{\text{data}} \| q_\theta)}_{\geq 0}$$

Since the entropy term does not depend on θ , minimizing the cross-entropy loss is equivalent to minimizing $D_{\text{KL}}(p^{\text{data}} \| q_\theta)$. By the non-negativity property of KL divergence, $D_{\text{KL}}(p^{\text{data}} \| q_\theta) \geq 0$ with equality if and only if $q_\theta = p^{\text{data}}$ almost everywhere. Therefore, the global minimizer θ^* satisfies:

$$\text{softmax}(\bar{\phi}_{\theta^*}(x)/\tau)_y = p^{\text{data}}(y | x) \quad \forall x, y$$

The model’s softmax output distribution matches the true data distribution. In practice, finite data and limited model capacity mean this equality holds only approximately, but the training objective drives q_θ toward p^{data} .

RNM with exponential noise does not sample from the softmax distribution. As shown in [Appendix B](#), it samples from the Permute-and-Flip (PF) distribution, whose probability mass function is (McKenna & Sheldon, 2020 [11]):

$$q(r | \bar{\phi}) = p_r \sum_{S \subseteq [V_k^+] \setminus \{r\}} \frac{(-1)^{|S|}}{|S| + 1} \prod_{s \in S} p_s, \quad p_j = \exp\left(\frac{\bar{\phi}_j - \bar{\phi}_{\max}}{\tau}\right) \quad \forall j \in V_k^+ \quad (1)$$

where $\tau = 2\Delta/\epsilon$ is the noise scale determined by the privacy budget.

The PF distribution concentrates more mass near the argmax token than softmax does at the same τ ([11]). This means that a model trained with $\mathcal{L}_{\text{CE}}$ produces logits $\bar{\phi}_\theta$ calibrated for softmax, but at inference these logits are decoded through the PF distribution, which sharpens the distribution beyond what we expect. The result is near-greedy token selection, which we want to fix.

6.2 MLE Under the PF Distribution

The fix is to train the model with the negative log-likelihood of the distribution it will actually decode with. Let $\{(x_{<t}^{(n)}, y_t^{(n)})\}_{n=1}^N$ denote the training corpus. The maximum likelihood estimator under the PF distribution solves:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}_{\text{PF}}(\bar{\phi}_{\theta}^{(n)}, y^{(n)}), \quad \mathcal{L}_{\text{PF}}(\bar{\phi}, y) = -\log q(y \mid \bar{\phi}) \quad (2)$$

By standard MLE consistency argument, the expected loss $\mathbb{E}_{(x,y) \sim p^{\text{data}}}[\mathcal{L}_{\text{PF}}]$ decomposes as:

$$\mathbb{E}[-\log q_{\theta}(y \mid x)] = H(p^{\text{data}}) + D_{\text{KL}}(p^{\text{data}} \parallel q_{\theta})$$

where $H(p^{\text{data}})$ is the entropy (constant in θ) and $D_{\text{KL}} \geq 0$ with equality if and only if $q_{\theta} = p^{\text{data}}$. Hence minimizing $\mathcal{L}_{\text{PF}}$ over θ drives the PF decoding distribution toward the data distribution.

6.3 Integral Identity and computationally feasible loss

Direct evaluation of (1) requires a sum over $2^{|V_k^+|-1}$ subsets, which is computationally infeasible for our current vocabulary sizes. We derive an integral form using the identity $\frac{1}{|S|+1} = \int_0^1 u^{|S|} du$. Substituting into (1):

$$\begin{aligned} q(r \mid \bar{\phi}) &= p_r \sum_{S \subseteq [V_k^+] \setminus \{r\}} (-1)^{|S|} \left(\int_0^1 u^{|S|} du \right) \prod_{s \in S} p_s \\ &= p_r \int_0^1 \sum_{S \subseteq [V_k^+] \setminus \{r\}} (-1)^{|S|} \prod_{s \in S} (p_s \cdot u) du \end{aligned} \quad (3)$$

$$= p_r \int_0^1 \prod_{s \neq r} (1 - p_s u) du \quad (4)$$

The exchange of sum and integral in (3) is justified because the sum is finite ($2^{|V_k^+|-1}$ terms) and all terms are bounded on $[0, 1]$ (since $0 < p_s \leq 1$ and $0 \leq u \leq 1$), so Fubini's theorem applies. The final step uses the binomial expansion $\sum_{S \subseteq A} (-1)^{|S|} \prod_{s \in S} (p_s u) = \prod_{s \in A} (1 - p_s u)$.

Taking the negative logarithm, the PF-MLE loss becomes:

$$\mathcal{L}_{\text{PF}}(\bar{\phi}, y) = -\frac{\bar{\phi}_y - \bar{\phi}_{\max}}{\tau} - \log \int_0^1 \prod_{s \neq y} (1 - p_s u) du \quad (5)$$

6.4 Gauss–Legendre Quadrature Approximation

We approximate the integral in (5) using K -point Gauss–Legendre quadrature on $[0, 1]$ with nodes $\{u_i\}_{i=1}^K$ and weights $\{w_i\}_{i=1}^K$. The resulting estimator is:

$$\hat{\mathcal{L}}_{\text{PF}}(\bar{\phi}, y) = -\frac{\bar{\phi}_y - \bar{\phi}_{\max}}{\tau} - \log \left(\sum_{i=1}^K \exp \left(\log w_i + \sum_{s \neq y} \log(1 - p_s u_i) \right) \right) \quad (6)$$

Computational cost. For each quadrature node u_i , the inner sum $\sum_{s \neq y} \log(1 - p_s u_i)$ requires $O(|V_k^+|)$ operations. With K nodes, the total cost is $O(K|V_k^+|)$ per token. This is linear in vocabulary size and comparable to cross-entropy. We use $K = 32$ throughout.

Numerical stability. The terms $\log(1 - p_s u_i)$ require care when $p_s u_i$ is close to 0 or 1. We used the `log1mexp` trick (Mächler, 2012) to prevent catastrophic cancellation and numerical overflow. All PF loss computations are performed in `float64` to maintain high precision.

6.5 Verification of the Quadrature Estimator

We verify $\hat{\mathcal{L}}_{\text{PF}}$ through the following checks on synthetic logit vectors drawn from $\mathcal{N}(0, 1)$:

1. **Quadrature vs. Monte Carlo simulation.** For a vocabulary of size $|V_k^+| = 10,000$ and the target token set to the second-highest logit, we compare the quadrature NLL against Monte Carlo simulation of the exponential noise RNM mechanism (5×10^5 trials) across $\tau \in \{0.9, 1.0, \dots, 1.5\}$. Figure 4 shows that the two estimates agree to within ~ 0.05 across all temperature values.
2. **Numerical gradient check.** We verify that the gradients computed by automatic differentiation match numerical approximations, using `torch.autograd.gradcheck` at $|V_k^+| = 100$ with tolerances `atol` = 10^{-4} , `rtol` = 10^{-3} . The check passes, confirming that the quadrature computation is differentiable and suitable for backpropagation during fine-tuning.

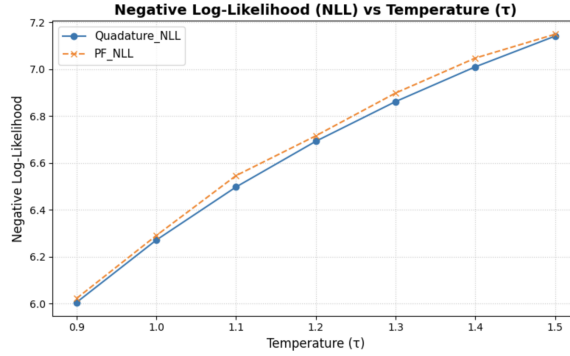


Figure 4: Verification of the quadrature NLL estimator ($K = 32$) against Monte Carlo simulation of the PF mechanism across varying temperature values

7 Fine-Tuning with QLoRA

We fine-tune Llama-3.2-3B-Instruct using QLoRA (Quantized Low-Rank Adaptation) with $\hat{\mathcal{L}}_{\text{PF}}$ replacing $\mathcal{L}_{\text{CE}}$ as the training objective. The fine-tuning is performed on the WikiText-103 training split, which serves as a general-purpose text corpus.

7.1 Quantization

The base model is loaded in 4-bit NormalFloat (NF4) quantization with double quantization enabled, reducing memory. All forward-pass computations are performed in `bfloat16` precision, while the PF loss is computed in `float64` to maintain numerical stability.

7.2 Low-Rank Adaptation

Instead of updating all 3.2 billion parameters, we use low-rank adapter matrices in the attention layers following the LoRA framework. The adapter is formulated as $W' = W + BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$ are the trainable low-rank matrices, W is the frozen pretrained weight, and r is the rank. We apply LoRA to the four attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`). This gives approximately 9.1 million trainable parameters, around 0.3% of the total model parameters.

We set the LoRA rank to $r = 16$ and the scaling factor to $\alpha = 32$ (i.e., $\alpha = 2r$), giving an effective learning rate multiplier of $\alpha/r = 2$ on the adapter weights. Dropout is set to 0 [13] since it provides no benefit in current setting.

7.3 Training Configuration

The PF temperature is set to $\tau = 1.1$, which matches the decoding temperature used in our InvisibleInk evaluation pipeline.

Training consists of 5000 optimizer steps with a batch size of 1 and gradient accumulation over 8 steps, giving an effective batch size of 8. We use AdamW with a learning rate of 10^{-4} , weight decay of 0.01, and a cosine learning rate schedule with 100 warmup steps. Gradient norms are clipped to 1.0. We use memory-efficient two-phase backward pass (PF Loss graph and model’s backward graph are not held simultaneously).

7.4 Convergence

We check the *alignment gap* on a held out validation set during training:

$$\text{gap} = \mathbb{E}[\hat{\mathcal{L}}_{\text{PF}}(\bar{\phi}_{\theta}, y)] - \mathbb{E}[\mathcal{L}_{\text{CE}}(\bar{\phi}_{\theta}, y)]$$

For a CE-trained model, this gap is positive since PF decoding assigns systematically lower probability to the true next token than softmax does. Fine-tuning with $\hat{\mathcal{L}}_{\text{PF}}$ reduces this gap. Training was stopped at step 2000 (out of a maximum of 5000), at which point the gap had converged to approximately -0.02 , indicating that the model’s logits are now calibrated for PF decoding.

The full set of hyperparameters is summarized in Table 1.

Table 1: QLoRA fine-tuning configuration.

Category	Parameter	Value
Model	Base model	Llama-3.2-3B-Instruct
	Quantization	4-bit NF4, double quantization
	Compute dtype	bfloat16
LoRA	Rank r	16
	Scaling α	32
	Target modules	q, k, v, o_proj
	Trainable parameters	$\sim 9.1\text{M}$ (0.3% of 3.2B)
PF Loss	Temperature τ	1.1
	Quadrature nodes K	32
Optimization	Optimizer	AdamW
	Learning rate	10^{-4} (cosine decay)
	Warmup steps	100
	Effective batch size	8 (1×8 accumulation)
	Positions per sequence	2 (random)
	Training steps	2000 (early stopped)
	Gradient clipping	1.0
Data	Corpus	WikiText-103 (train split)
	Sequence length	256
Hardware	GPU	NVIDIA A100 (80 GB VRAM)

8 Verification of Utility Recovery through the new Loss function

We used a similar setup as explained in Section 5 and added a new mechanism called ‘Invisible Ink + RNM + PF’:

Invisible Ink + RNM + PF: This is a similar setting to the Invisible Ink + RNM mechanism but the model used is our fine tuned version of Llama-3.2-3B-Instruct with the PF MLE ($\hat{\mathcal{L}}_{\text{PF}}$) loss function.

Since we have also established in Section 5 that MAUVE scores are a much better metric than Perplexity, we will compare the MAUVE scores of the 4 different mechanisms (Base Invisible Ink, Weak Invisible Ink, Invisible Ink + RNM, Invisible Ink + RNM + PF), as shown in Figure 5, and we can observe that Invisible Ink + RNM + PF has the best MAUVE scores (utility) out of all the mechanisms.

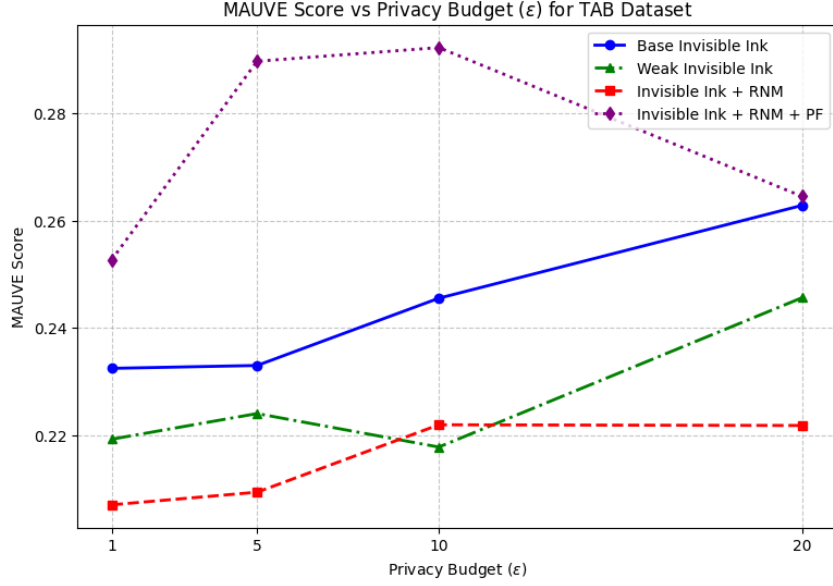


Figure 5: MAUVE scores of 500 samples each of Base Invisible Ink, Weak Invisible Ink, Invisible Ink + RNM, Invisible Ink + RNM + PF for TAB dataset

References

- [1] Vishnu Vinod, Krishna Pillutla, and Abhradeep Thakurta. InvisibleInk: High-Utility and Low-Cost Text Generation with Differential Privacy. In NeurIPS, 2025. <https://arxiv.org/abs/2507.02974>
- [2] Kareem Amin, Alex Bie, Weiwei Kong, Alexey Kurakin, Natalia Ponomareva, Umar Syed, Andreas Terzis, and Sergei Vassilvitskii. Private prediction for large-scale synthetic text generation. In Proc. of EMNLP Findings, 2024. <https://arxiv.org/abs/2407.12108>
- [3] Jinshuo Dong, David Durfee, Ryan Rogers. Optimal Differential Privacy Composition for Exponential Mechanisms and the Cost of Adaptivity. arXiv:1909.13830. <https://arxiv.org/abs/1909.13830>
- [4] Iris A. M. Huijben, Wouter Kool, Max B. Paulus, Ruud J. G. van Sloun. A Review of the Gumbel-max Trick and its Extensions for Discrete Stochasticity in Machine Learning. arXiv:2110.01515. <https://arxiv.org/abs/2110.01515>
- [5] Report Noisy Max Mechanisms. <https://docs.opendp.org/en/stable/api/user-guide/measurements/noisy-max-mechanisms.html>
- [6] Natalia Ponomareva, Hussein Hazimeh, Alex Kurakin, Zheng Xu, Carson Denison, H Brendan McMahan, Sergei Vassilvitskii, Steve Chien, and Abhradeep Guha Thakurta. How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy. Journal of Artificial Intelligence Research, 2023. <https://arxiv.org/abs/2303.00654>
- [7] Jonathan Lebensold, Doina Precup, Borja Balle. On the Privacy of Selection Mechanisms with Gaussian Noise. arXiv:2402.06137. <https://arxiv.org/abs/2402.06137>
- [8] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. Mauve: Measuring the gap between neural text and human text using divergence frontiers. In Proc. of NeurIPS, 2021. <https://arxiv.org/abs/2102.01454>
- [9] Krishna Pillutla, Lang Liu, John Thickstun, Sean Welleck, Swabha Swayamdipta, Rowan Zellers, Sewoong Oh, Yejin Choi, and Zaid Harchaoui. MAUVE Scores for Generative Models: Theory and Practice. JMLR, 2023. <https://arxiv.org/abs/2212.14578>
- [10] Shenglai Zeng, Jiankun Zhang, Pengfei He, Yue Xing, Yiding Liu, Han Xu, Jie Ren, Shuaiqiang Wang, Dawei Yin, Yi Chang, and Ziliang Tang. Exploring Privacy Issues in Retrieval-Augmented Generation (RAG). In ACL 2024. <https://arxiv.org/abs/2402.16893>
- [11] Ryan McKenna and Daniel Sheldon. Permute-and-Flip: A new mechanism for differentially private selection. In NeurIPS, 2020. <https://arxiv.org/abs/2010.12603>

[12] Meta AI. Llama 3.2: Lightweight Models for Edge Devices. 2024. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>

[13] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized Language Models. In NeurIPS, 2023. <https://arxiv.org/abs/2305.14314>

A Equivalence of the Gumbel-Max Mechanism and the Exponential Mechanism

We are given a $|V_k^+|$ -dimensional vector of average clipped logits $\bar{\phi} = (\bar{\phi}_1, \dots, \bar{\phi}_{|V_k^+|})$ where V_k^+ represents the Expanded Top-K+ Vocabulary [1]. To ensure differential privacy, we inject noise into these logits before selecting the next token. We formalize the proof that adding independent Gumbel noise to each logit and selecting the argmax is probabilistically equivalent to sampling from the Exponential Mechanism.

Gumbel Distribution: A random variable Z follows a Gumbel distribution with location parameter μ and scale parameter b , denoted as $Z \sim \text{Gumbel}(\mu, b)$, if its cumulative distribution function (CDF) is given by:

$$F(z) = \exp\left(-\exp\left(-\frac{z-\mu}{b}\right)\right)$$

and its probability density function (PDF) is:

$$f(z) = \frac{1}{b} \exp\left(-\frac{z-\mu}{b}\right) \exp\left(-\exp\left(-\frac{z-\mu}{b}\right)\right)$$

To establish the main result, we first prove two foundational lemmas regarding the properties of Gumbel random variables.

Lemma 1: Stability of the Maximum of Gumbel Variables

The maximum of k independent Gumbel random variables with the same scale parameter b is also a Gumbel random variable with scale b .

Proof. Let $Z_1, Z_2, \dots, Z_k$ be independent random variables such that $Z_i \sim \text{Gumbel}(\mu_i, b)$. Define $Z_{\max} = \max_{i \in \{1, \dots, k\}} Z_i$. The CDF of $Z_{\max}$ is evaluated as follows:

$$\begin{aligned} P(Z_{\max} \leq z) &= \prod_{i=1}^k P(Z_i \leq z) \\ &= \prod_{i=1}^k \exp\left(-\exp\left(-\frac{z-\mu_i}{b}\right)\right) \end{aligned}$$

Using the property of exponents $\prod \exp(x_i) = \exp(\sum x_i)$, we can rewrite this as:

$$P(Z_{\max} \leq z) = \exp\left(-\sum_{i=1}^k \exp\left(-\frac{z-\mu_i}{b}\right)\right)$$

Factoring out the term dependent on z :

$$P(Z_{\max} \leq z) = \exp\left(-\exp\left(-\frac{z}{b}\right) \sum_{i=1}^k \exp\left(\frac{\mu_i}{b}\right)\right)$$

By utilizing the identity $x = \exp(\ln x)$ for the summation term, we obtain:

$$\begin{aligned} P(Z_{\max} \leq z) &= \exp\left(-\exp\left(-\frac{z}{b}\right) \exp\left(\ln \sum_{i=1}^k \exp\left(\frac{\mu_i}{b}\right)\right)\right) \\ &= \exp\left(-\exp\left(-\frac{z - b \ln\left(\sum_{i=1}^k \exp\left(\frac{\mu_i}{b}\right)\right)}{b}\right)\right) \end{aligned}$$

This is precisely the CDF of a Gumbel distribution. Therefore, $Z_{\max} \sim \text{Gumbel}(\mu^*, b)$, where the new location parameter is $\mu^* = b \ln\left(\sum_{i=1}^k \exp\left(\frac{\mu_i}{b}\right)\right)$. ■

Lemma 2: Pairwise Probability of Gumbel Variables

Given two independent Gumbel random variables $Z_i \sim \text{Gumbel}(\mu_i, b)$ and $Z_j \sim \text{Gumbel}(\mu_j, b)$, the probability that Z_i strictly exceeds Z_j is given by the softmax function of their location parameters scaled by b .

Proof. We calculate the probability $P(Z_i > Z_j)$ by integrating over all possible values k of Z_j :

$$P(Z_i > Z_j) = \int_{-\infty}^{\infty} P(Z_j < k) f_{Z_i}(k) dk$$

Substituting the CDF of Z_j and the PDF of Z_i :

$$P(Z_i > Z_j) = \int_{-\infty}^{\infty} \exp\left(-\exp\left(-\frac{k - \mu_j}{b}\right)\right) \frac{1}{b} \exp\left(-\frac{k - \mu_i}{b}\right) \exp\left(-\exp\left(-\frac{k - \mu_i}{b}\right)\right) dk$$

Combining the double-exponential terms:

$$P(Z_i > Z_j) = \int_{-\infty}^{\infty} \frac{1}{b} e^{-k/b} e^{\mu_i/b} \exp\left(-e^{-k/b} (e^{\mu_j/b} + e^{\mu_i/b})\right) dk$$

We apply the substitution $u = e^{-k/b}$, which yields $du = -\frac{1}{b} e^{-k/b} dk$. The integration limits change from $(-\infty, \infty)$ to $(\infty, 0)$:

$$P(Z_i > Z_j) = e^{\mu_i/b} \int_0^{\infty} \exp\left(-u (e^{\mu_j/b} + e^{\mu_i/b})\right) du$$

Evaluating the integral:

$$P(Z_i > Z_j) = e^{\mu_i/b} \left[\frac{\exp(-u (e^{\mu_j/b} + e^{\mu_i/b}))}{-(e^{\mu_j/b} + e^{\mu_i/b})} \right]_0^{\infty}$$

At $u \rightarrow \infty$, the term approaches 0. At $u = 0$, the numerator is 1.

$$P(Z_i > Z_j) = e^{\mu_i/b} \left(\frac{1}{e^{\mu_j/b} + e^{\mu_i/b}} \right) = \frac{\exp(\mu_i/b)}{\exp(\mu_i/b) + \exp(\mu_j/b)}$$

■

Main Theorem: Equivalence to the Exponential Mechanism

Let $Z_1, Z_2, \dots, Z_{|V_k^+|}$ represent the noisy logits, where $Z_j \sim \text{Gumbel}(\mu_j, b)$ for all $j \in \{1, \dots, |V_k^+|\}$. We wish to find the probability that token i is selected, which occurs when Z_i is the maximum value among all logits.

Proof. Let us define a composite random variable representing the maximum of all logits excluding i :

$$Z_{-i} = \max_{j \neq i} Z_j$$

By **Lemma 1**, we know that $Z_{-i} \sim \text{Gumbel}(\mu_{-i}, b)$, where the location parameter μ_{-i} satisfies:

$$\exp\left(\frac{\mu_{-i}}{b}\right) = \sum_{j \neq i} \exp\left(\frac{\mu_j}{b}\right)$$

We can now reduce the d -dimensional argmax problem to a pairwise comparison between Z_i and Z_{-i} . Applying **Lemma 2**:

$$\begin{aligned} P(\text{token } i \text{ is selected}) &= P(Z_i > Z_{-i}) \\ &= \frac{\exp(\mu_i/b)}{\exp(\mu_i/b) + \exp(\mu_{-i}/b)} \end{aligned}$$

Substituting the expression for $\exp(\mu_{-i}/b)$:

$$\begin{aligned} P(\text{token } i \text{ is selected}) &= \frac{\exp(\mu_i/b)}{\exp(\mu_i/b) + \sum_{j \neq i} \exp(\mu_j/b)} \\ &= \frac{\exp(\mu_i/b)}{\sum_{j=1}^{|V_k^+|} \exp(\mu_j/b)} \end{aligned}$$

This final probability distribution is exactly the standard definition of the Exponential Mechanism, where elements are sampled with probability proportional to $\exp(\text{score}/b)$, thus completing the proof.

B Equivalence of Permute-and-Flip and Report Noisy Max

This is an attempt to try to show the theoretical equivalence between the Permute and Flip mechanism by bridging the two methods using a sequence of intermediate mechanisms. The noise parameter is $\lambda = \frac{2\Delta}{\epsilon}$ which ensures ϵ -DP. Let $\bar{\phi}^* = \max_{j \in V_+^k} \bar{\phi}(j, R)$ denote the true maximum score over the domain V_+^k and the noise variables as $Z_i \sim \text{Exp}(\lambda)$.

B.1 Algorithm Definitions

For the proof we define two intermediate algorithms A and B.

- **Algorithm 1 (RNM):** For each $i \in V_+^k$, compute $v_i = \bar{\phi}(i, R) + Z_i$. Output $\arg \max_i v_i$.
- **Algorithm 2 (P&F):** Randomly permute V_+^k . For each i in the permuted order, return i with probability $\exp(-(\frac{\bar{\phi}^* - \bar{\phi}(i, R)}{\lambda}))$. Output the first index that succeeds.
- **Algorithm 3 (Intermediate A):** For each $i \in V_+^k$, compute $v_i = \bar{\phi}(i, R) + Z_i$. Define the candidate set $S = \{i : v_i \geq \bar{\phi}^*\}$. Output an element from S uniformly at random.
- **Algorithm 4 (Intermediate B):** For each $i \in V_+^k$, compute the capped score $v_i^\dagger = \min(\bar{\phi}^*, \bar{\phi}(i, R) + Z_i)$. Define $S' = \{i : v_i^\dagger = \bar{\phi}^*\}$. Output $\arg \max_{i \in S'} (\bar{\phi}^* + Z'_i)$, where $Z'_i \sim \text{Exp}(\lambda)$ is fresh noise.

B.2 Equivalence Proofs

Lemma 1. *The output distributions of Algorithm 2 and Algorithm 3 are identical.*

Proof. In Algorithm 3, an index i enters the set S if the additive noise Z_i is sufficiently large so that the maximum score $\bar{\phi}^*$ reaches. This gives us that the probability of this event is:

$$\mathbb{P}(i \in S) = \mathbb{P}(Z_i \geq \bar{\phi}^* - \bar{\phi}(i, R)) = \exp(-(\frac{\bar{\phi}^* - \bar{\phi}(i, R)}{\lambda}))$$

This exactly matches the Bernoulli trial probability in Algorithm 2. Algorithm 2 evaluates candidates in a uniformly random permutation and selects the first winner, it is statistically equivalent to uniformly sampling one value from the set S of all successes of the possible values. $\square$

Lemma 2. *The output distributions of Algorithm 3 and Algorithm 4 are identical.*

Proof. Both algorithms identify the same candidate set ($S = S'$). Algorithm 3 samples $i \in S$ uniformly at random, while Algorithm 4 computes $\arg \max_{i \in S'} (\bar{\phi}^* + Z'_i)$. Since $\bar{\phi}^*$ is constant and Z'_i are continuous, i.i.d. random variables, this argmax yields a strictly uniform distribution over S' . Therefore, both tie-breaking mechanisms are identical in distribution. $\square$

Lemma 3. *The output distributions of Algorithm 4 and Algorithm 1 are identical.*

Proof. Let $v_i^\dagger$ represent the final effective score for candidate i in Algorithm 4

$$v_i^\dagger = \begin{cases} \bar{\phi}(i, R) + Z_i & \text{if } \bar{\phi}(i, R) + Z_i < \bar{\phi}^* \\ \bar{\phi}^* + Z'_i & \text{if } \bar{\phi}(i, R) + Z_i \geq \bar{\phi}^* \end{cases}$$

For any threshold $t \geq \bar{\phi}^*$, the probability that the final effective score for token i exceeds t is the joint probability that its initial noisy score reaches the cap and the subsequent fresh noise spans the remaining distance to t :

$$\begin{aligned} \mathbb{P}(v_i^\dagger > t) &= \mathbb{P}(Z_i \geq \bar{\phi}^* - \bar{\phi}(i, R)) \cdot \mathbb{P}(Z'_i > t - \bar{\phi}^*) \\ &= \exp(-\lambda(\bar{\phi}^* - \bar{\phi}(i, R))) \cdot \exp(-\lambda(t - \bar{\phi}^*)) \\ &= \exp(-\lambda(t - \bar{\phi}(i, R))) \end{aligned}$$

This shows that the two-stage capped noise process in Algorithm 4 is distributionally equivalent to the single noise generation in Algorithm 1. Because both algorithms return the argument that maximizes these distributionally equivalent scores, their final outputs are identical in distribution. $\square$

Theorem 1. *The Permute-and-Flip mechanism (Algorithm 2) and the Report Noisy Max mechanism with Exponential noise (Algorithm 1) possess identical output distributions.*

Proof. The proof follows directly from the sequential application of Lemma 1, Lemma 2, and Lemma 3. $\square$

C MAUVE score

Let P represent the true probability distribution of human-written reference text, and Q represent the probability distribution of the machine-generated text. To capture the trade-off between Type I errors (where the model generates unnatural or degenerate text) and Type II errors (where the model fails to capture the diversity of human text), MAUVE computes the Kullback-Leibler (KL) divergence against a mixture distribution.

For a mixing parameter $\lambda \in (0, 1)$, the mixture distribution is defined as:

$$R_\lambda = \lambda P + (1 - \lambda)Q$$

The divergence frontier $\mathcal{F}(P, Q)$ is constructed by mapping the trade-off curve across all values of λ :

$$\mathcal{F}(P, Q) = \{(\exp(-\text{KL}(Q \parallel R_\lambda)), \exp(-\text{KL}(P \parallel R_\lambda))) \mid \lambda \in (0, 1)\} \cup \{(1, 0), (0, 1)\}$$

The MAUVE score is defined as the Area Under the Curve (AUC) of this divergence frontier. Mathematically, it can be expressed as the following integral:

$$\text{MAUVE}(P, Q) = \int_{\mathcal{F}(P, Q)} y \, dx$$

Expressed explicitly in terms of the parameter λ (assuming $x(\lambda)$ is monotonically increasing or integrating along the path):

$$\text{MAUVE}(P, Q) = \int_0^1 y(\lambda) \left| \frac{dx(\lambda)}{d\lambda} \right| d\lambda$$

Substituting the full KL divergence terms, the integral becomes:

$$\text{MAUVE}(P, Q) = \int_0^1 \exp(-\text{KL}(P \parallel R_\lambda)) \left| \frac{d}{d\lambda} \exp(-\text{KL}(Q \parallel R_\lambda)) \right| d\lambda$$

It provides a single scalar value bounded between 0 and 1, where a higher score indicates that the machine-generated text distribution Q is highly statistically similar to the human-written text distribution P .

D Perplexity and Perplexity Difference

For a generated sequence $X = (x_1, x_2, \dots, x_N)$ and a language model parameterized by θ , the perplexity is defined as:

$$\text{PPL}(X) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P_\theta(x_i \mid x_{<i}) \right)$$

where $P_\theta(x_i \mid x_{<i})$ is the probability assigned by the model to the i -th token given all preceding tokens.

Let X_{priv} represent the text generated using the private decoding mechanism (e.g., RNM or Exponential mechanism) and X_{pub} represent the text generated using standard non-private decoding (e.g., standard softmax sampling). The perplexity difference is given by:

$$\Delta \text{PPL} = \text{PPL}(X_{\text{priv}}) - \text{PPL}(X_{\text{pub}})$$

A lower ΔPPL indicates that the private generation mechanism successfully preserves the linguistic quality and fluency of the underlying non-private language model.